

2] Explain the difference between INSERT, UPDATE, and DELETE triggers.

Ans:

Difference Between INSERT, UPDATE, and DELETE Triggers in SQL

Triggers are SQL procedures that automatically execute when specific events occur on a table or view. The INSERT, UPDATE, and DELETE triggers are designed to respond to different types of data modification operations. Here's how they differ:

1. INSERT Trigger:

Purpose: An INSERT trigger is activated when a new row is inserted into a table.

When It's Used: To perform actions or validation before or after inserting a new record (e.g., logging, data validation, or cascading insertions).

Behavior: The trigger runs when the INSERT operation is executed on the table.

It can be used to ensure that only valid data is inserted into the table or to trigger actions that need to happen when a new row is added.

Example:

```
CREATE TRIGGER before_employee_insert
BEFORE INSERT ON employees
FOR EACH ROW
BEGIN
    IF NEW.salary < 0 THEN
```

```
SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Salary cannot be negative';
```

```
END IF;
```

```
END;
```

2. UPDATE Trigger:

Purpose: An UPDATE trigger is fired when an existing row in the table is updated (modified).

When It's Used: To track changes made to data (e.g., auditing changes or logging modifications).

To perform actions before or after the update, such as maintaining consistency between related tables or validating updated data.

Behavior: The trigger runs when the UPDATE operation is executed on the table.

It can reference both the old and new values of the updated row, allowing you to compare them and act accordingly.

Example:

```
CREATE TRIGGER after_salary_update
```

```
AFTER UPDATE ON employees
```

```
FOR EACH ROW
```

```
BEGIN
```

```
    INSERT INTO salary_log (employee_id, old_salary, new_salary,  
change_date)
```

```
    VALUES (OLD.employee_id, OLD.salary, NEW.salary, NOW());
```

```
END;
```

3. DELETE Trigger:

Purpose: A DELETE trigger is activated when a row is deleted from a table.

When It's Used: To handle data cleanup, logging, or to ensure referential integrity when rows are deleted.

To prevent accidental or unauthorized deletions by either rolling back the operation or logging it.

Behavior: The trigger runs when the DELETE operation is executed on the table.

It can reference the old values of the deleted row to, for example, log the deleted data before it is removed.

Example:

```
CREATE TRIGGER before_employee_delete
BEFORE DELETE ON employees
FOR EACH ROW
BEGIN
    INSERT INTO deletion_log (employee_id, deleted_at)
    VALUES (OLD.employee_id, NOW());
END;
```